Лабораторная работа 1.14. Создание новых записей в базе данных

Цель лабораторной работы.

Получить практические навыки в разработке для веб-приложения операций создания новых записей в базе данных.

Введение

Для разработки кода веб-приложения, реализующего операции вставки новых сущностей в базу данных, необходимо для проекта создать ветку dev6 на основе ветки master. После проверки преподавателем работоспособности веб-приложения ветка dev6 должна быть слита с веткой master.

Создание новых данных в базе данных

Создание новой должности

Для создания новой должности и сохранении этой информации в базе данных используется форма страницы role.jps.

```
<form method="POST" action="">
      <h3>Новая должность</h3>
      <div class="mb-3">
            <br> <label for="inputRole"
              class="col-sm-3 col-form-label">Должность</label>
            <div class="col-sm-6">
                  <input type="text" name="inputRole"
                        class="form-control" id="personRole" />
            </div>
      </div>
      <p>
            <br> <br> <br>
            <button type="submit"
                  class="btn btn-primary">Добавить</button>
            <br>
      </p>
</form>
```

При создании новой должности пользователь должен ввести в поле inputRole новую должность и нажать кнопку Добавить.

Логика добавления новой должности реализована в сервлете RoleServlet.

Переопределим метод doPost( ) сервлета RoleServlet.

```
protected void doPost(HttpServletRequest request, HttpServletResponse
response)
                throws ServletException, IOException {

          RoleDbDAO dao = new RoleDbDAO();

          String name = request.getParameter("inputRole");
          Role newRole = new Role(name);

          try {
                Long index = dao.insert(newRole);
                System.out.println("Adding result: " + index );
          } catch (DAOException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
          }
          doGet(request, response);
     }
```

Создаем контекст соединения с базой данных *dao:*

```
RoleDbDAO dao = new RoleDbDAO();
```

Параметру *name* присвоим значение новой должности, полученное из представления role.jsp методом getParameter.

```
String name = request.getParameter("inputRole");
```

Создадим экземпляр нового класса Role – newRole.

```
Role newRole = new Role(name);
```

Далее вызываем метод insert( ) для вставки новой сущности в базу данных roles.

```
Long index = dao.insert(newRole);
```

В завершении вставки новой сущности в базу данных вызываем метод doGet( ) для обновления отображаемого в браузере списка должностей

```
     doGet(request, response);
```

Запустим приложение и перейдем на страницу Должности. Для добавления новой должности в правой части страницы введем должность «водитель» (рис. 1).

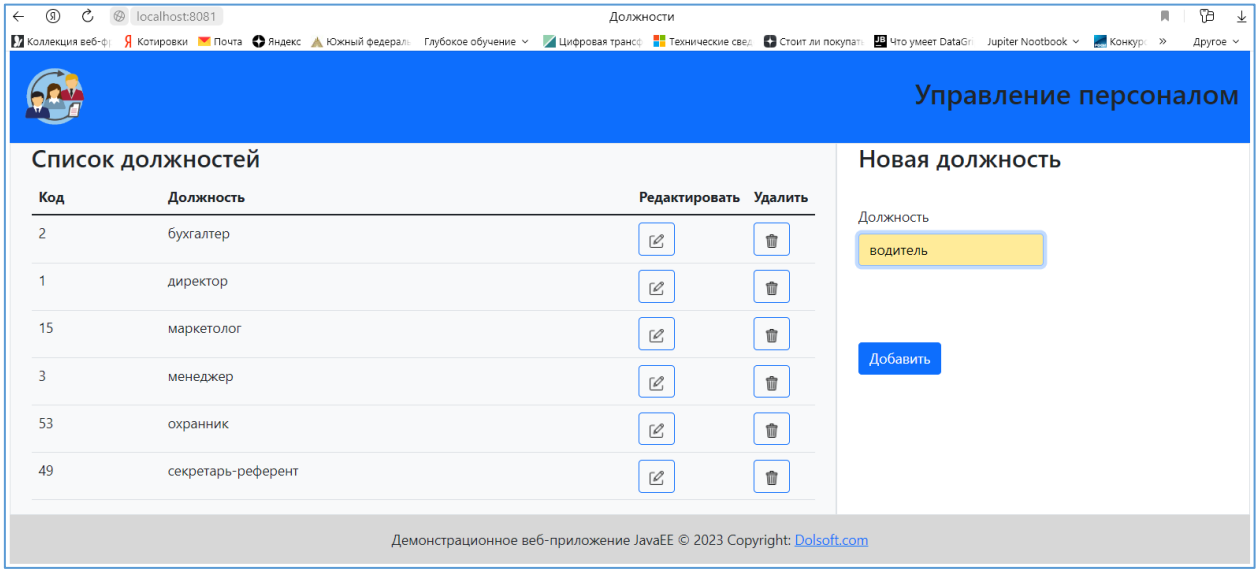


Рисунок 1 – Ввод новой должности

После нажатия кнопки «Добавить» в базе данных в таблице roles будет добавлена новая запись (рис. 2). При этом будет сгенерировано значение первичного ключа id = 54.

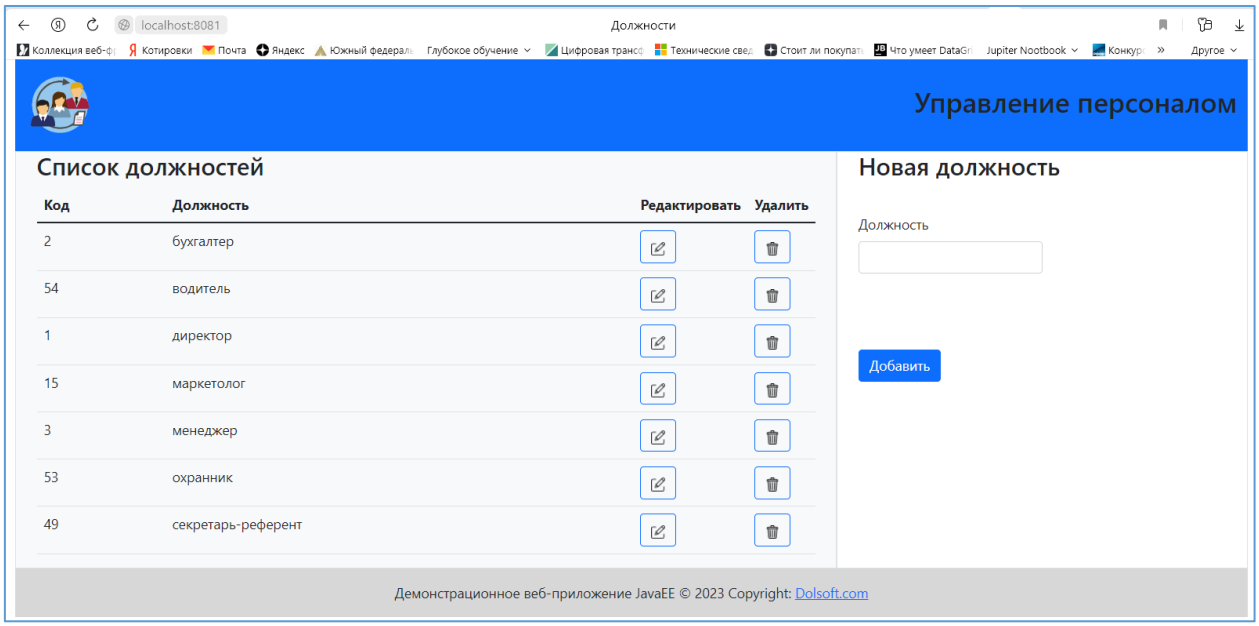


Рисунок 2 – Отображение новой должности

Следует отметить, что значение первичного ключа формируется в базе данных PostgreSQL, путем увеличения на 1 внутреннего счетчика базы. Так, если после строки с id равным 53 проводились операции добавления и удаления данных, то счетчик первичного ключа будет увеличиваться после каждой операции добавления данных, даже если операция была неудачной. Это объясняет генерируемое значение id, равное 54, в нашем случае при выполнении операции вставки новой должности – водитель.

Создание нового сотрудника

Для создания нового сотрудника и сохранении этой информации в базе данных используется форма страницы persons.jps.

```
<form method="POST" action="">
      <h3>Новый сотрудник</h3>
      <br>
      <div class="mb-3 row">
      <label for="lastname"
            class="col-sm-3 col-form-label">Фамилия</label>
      <div class="col-sm-7">
      <input type="text" class="form-control" id="staticLastname"
            name="lastname" />
      </div>
      </div>
      <div class="mb-3 row">
      <label for="firstname"
            class="col-sm-3 col-form-label">Имя</label>
      <div class="col-sm-7">
      <input type="text" class="form-control" id="staticFirstname"
      name="firstname" />
      </div>
      </div>
      <div class="mb-3 row">
      <label for="rolename" class="col-sm-3 col-form-
label">Должность</label>
      <div class="col-sm-7">
      <select name="role" class="form-control">
      <option>Выберите должность</option>
      <c:forEach var="role" items="${roles}">
      <option value="${role}">
      <c:out value="${role.getNamerole()}"></c:out>
      </option>
      </c:forEach>
      </select>
```

```
        </div>
        </div>
        <div class="mb-3 row">
        <label for="phone"
             class="col-sm-3 col-form-label">Телефон</label>
        <div class="col-sm-7">
        <input type="text" class="form-control" id="staticphone"
             name="phone" />
        </div>
        </div>
        <div class="mb-3 row">
        <label for="email" class="col-sm-3 col-form-label">Эл.почта
        </label>
        <div class="col-sm-7">
        <input type="text" class="form-control" id="staticemail"
             name="email" />
        </div>
        </div>
        <p>
        <br>
        <button type="submit" class="btn btn-primary">Добавить</button>
        </p>
</form>
```

При создании данных для нового сотрудника пользователь должен ввести необходимые данные по сотруднику, при этом должность выбирается из заданного списка. После ввода данных необходимо нажать кнопку «Добавить».

Логика добавления нового сотрудника реализована в сервлете PersonsServlet.

Переопределим метод doPost( ) сервлета PersonsServlet.

```
package controller;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.List;
```

```
import dao.ConnectionProperty;
import dao.PersonDbDAO;
import dao.RoleDbDAO;

import domain.Person;
import domain.Role;
import exception.DAOException;

@WebServlet("/person")
public class PersonServlet extends HttpServlet {

      private static final long serialVersionUID = 1L;
      ConnectionProperty prop;
    public PersonServlet() throws FileNotFoundException, IOException {
        super();
         prop = new ConnectionProperty();
    }

      protected void doGet(HttpServletRequest request,
HttpServletResponse response)
                  throws ServletException, IOException {

            response.setContentType("text/html");
            String userPath;
            List<Person> persons;
            List<Role> roles;

            RoleDbDAO daoRole = new RoleDbDAO();
            PersonDbDAO dao = new PersonDbDAO();

            try {
                  roles = daoRole.findAll();
                  request.setAttribute("roles", roles);
            } catch (DAOException e) {
                  // TODO Auto-generated catch block
                  e.printStackTrace();
            }


            try {
                  persons = dao.findAll();
                  roles = daoRole.findAll();
                  for (Person person: persons) {

      person.setRole(daoRole.FindById(person.getIdRole(), roles));
                  }

                  request.setAttribute("persons", persons);

            } catch (DAOException e) {
                  // TODO Auto-generated catch block
```

```
                e.printStackTrace();
            }

          userPath = request.getServletPath();
            if("/person".equals(userPath)){

      request.getRequestDispatcher("/views/person.jsp").forward(request
, response);
            }
      }
      protected void doPost(HttpServletRequest request,
HttpServletResponse response)
                throws ServletException, IOException {

            PersonDbDAO dao = new PersonDbDAO();

            String firstName = request.getParameter("firstname");
            String lastName = request.getParameter("lastname");
            String email = request.getParameter("email");
            String phone = request.getParameter("phone");

            String role =  request.getParameter("role");

            int index1 = role.indexOf('=');
            int index2 = role.indexOf(",");
            String r1 = role.substring(index1+1, index2);
            Long idRole = Long.parseLong(r1.trim());

            Person newPerson = new Person(idRole, firstName, lastName,
phone, email, idRole) ;

            try {
                  Long index = dao.insert(newPerson);
                  System.out.println("Adding result: " + index );
            } catch (DAOException e) {
                  // TODO Auto-generated catch block
                  e.printStackTrace();
            }
            doGet(request, response);
      }
}
```

Логика работы метода doPost( ) сервлета PersonsServlet аналогична работе метода doPost( ) сервлета RolesServlet, за исключением некоторых дополнений.

В форме при создании данных нового сотрудника необходимо иметь список всех возможных должностей. Для этого в методе doPost( ) требуется загрузить список должностей roles.

```
RoleDbDAO daoRole = new RoleDbDAO();
try {
      roles = daoRole.findAll();
      request.setAttribute("roles", roles);
      } catch (DAOException e) {
            e.printStackTrace();
      }
}
```

При выборе должности сотрудника на странице persons.jsp возвращаемым значением блока <select /> является экземпляр класса Role. Для получения параметра role в сервлете RolesServlet используется метод request.getParameter( ).

```
String role = request.getParameter("role");
```

Данный метод возвращает строковое значение экземпляра класса Roles:

```
Role {Id = 49, NameRole = секретарь-референт}
```

Из этой строки необходимо выделить значение параметра Id, которое требуется задать для параметра roleId. Для этого используется следующий фрагмент кода.

```
String role = request.getParameter("role");
int index1 = role.indexOf('=');
int index2 = role.indexOf(",");
String r1 = role.substring(index1+1, index2);
Long roleId = Long.parseLong(r1.trim());
```

Перейдем с главной страницы веб-приложения на страницу Сотрудники. Для добавления данных нового сотрудника в правой части страницы введем необходимые параметры (рис. 3).

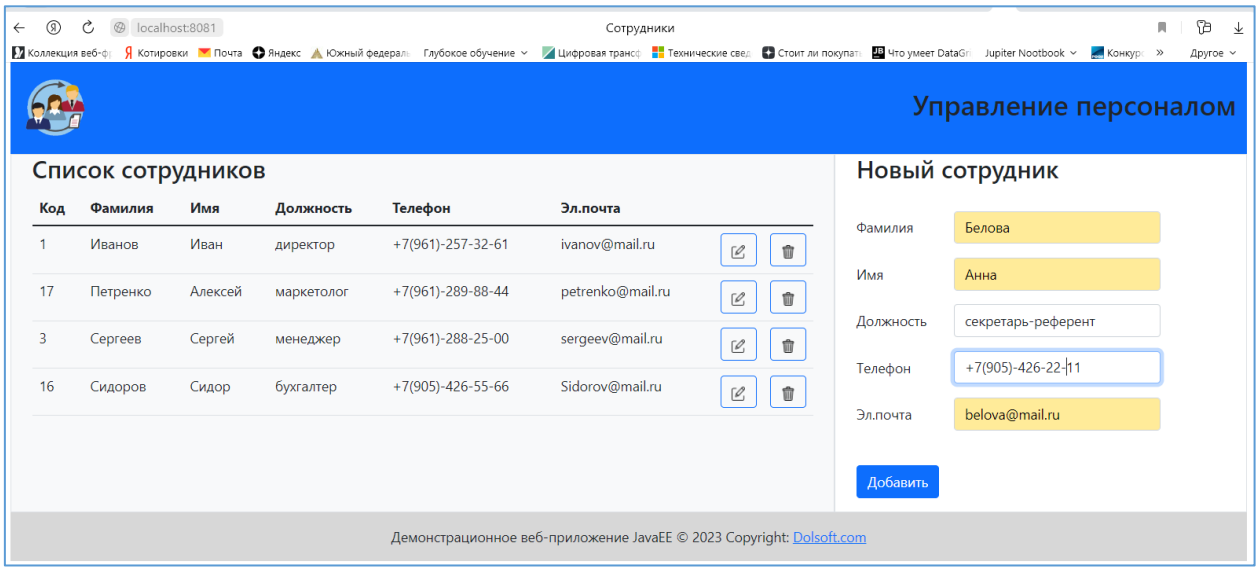


Рисунок 3 – Ввод данных нового сотрудника

После нажатия кнопки Добавить в базе данных в таблице persons будет добавлена новая запись (рис. 4). При этом будет сгенерировано значение первичного ключа id.

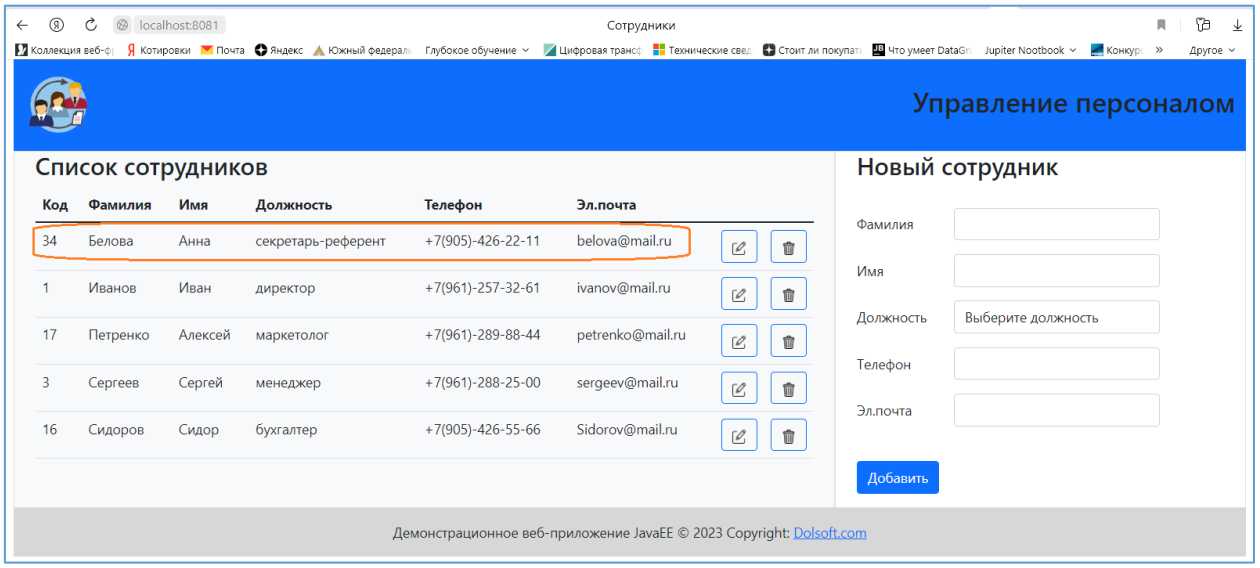


Рисунок 4 – Отображение нового сотрудника

Задание на лабораторную работу

1. Изучить теоретический материал лекции 1.6 – 1.8.
2. По варианту, определенному преподавателем, разработать переопределение методов doPost( ) для сервлетов, обеспечивающих ввод новых данных в таблицы базы данных.

3. Продемонстрировать результаты работы веб-приложения предателю.
4. Слить ветку разработки dev6 с основной веткой master.
5. Подготовить отчет по лабораторной работе и загрузить его на учебный портал.